Admission Control for LLM Inference with Heavy-Tailed Request Costs: Coarse Predictions Suffice for Ordering, Not for Budgets

Suman Babu

Abstract

LLM output lengths are heavy-tailed and only partly predictable from the prompt. A handful of long generations can tie up every concurrency slot at an inference gateway, leaving short interactive requests stuck in queue. We ask which scheduling and admission policy best protects p99 latency in this setting.

We model the gateway as a multi-server queue with non-preemptive dispatch, heavy-tailed service times, and a per-backend KV-cache budget, then compare three approaches: FIFO with a concurrency limit, shortest-predicted-job-first (SPJF) with aging, and token-budget admission (TBA) with a reserved partition for short requests. Multi-server heavy-tail theory tells us that FIFO's waiting-time tail index is $[c - \rho](\alpha - 1)$, meaning throughput-tuned concurrency limits work against p99 performance. Modeling prediction error as log-normal noise of scale σ yields three design rules: a closed-form misordering probability, a leakage limit for the protected partition, and a crossover σ beyond which budgets waste more capacity than a simple count limit.

In a discrete-event simulation with Pareto ($\alpha = 1.5$) output lengths at load $\rho = 0.85$, combining SPJF ordering with TBA cuts p99 time-to-first-token for short requests from 22.8 s to 3.2 s—a $7.1\times$ improvement. P99 slowdown drops from 11.8 to 2.1, and engine preemptions fall from 2.79 to 0.05 per 1,000 requests, while goodput climbs from 87.6% to 96.7%.

The ordering gain holds up well: SPJF alone keeps a 1.8–$2.5\times$ improvement from $\sigma = 0$ to 1.5. The budget gain does not: the hybrid loses its edge over SPJF alone at $\sigma = 1$ and falls below FIFO's goodput at $\sigma = 1.5$—sooner than the analytic crossover predicts. A predictor-free, age-only budget adds nothing. Long requests pay for the protection with 22% higher p99 TTFT.

Keywords: LLM inference, admission control, heavy-tailed service times, shortest-job-first, scheduling with predictions, KV cache, tail latency, goodput, API gateway.

Introduction

Web requests are roughly interchangeable in cost. LLM requests are not. One prompt produces 20 tokens; the next produces 3,000. The gateway in front of the model servers has no way to know the output length upfront. Output lengths follow a heavy-tailed distribution: most requests are short, but a small fraction run very long. A few of those long generations can hold every concurrency slot, forcing the short interactive

requests—the majority of traffic—to wait. Users experience this as slow time-to-first-token (TTFT), which is the latency number that matters most for interactive products.

This is a classic heavy-tail queueing problem with two LLM-specific wrinkles. First, memory—not compute—is the binding resource. Each in-flight request holds KV-cache memory that grows with every token it generates. A count limit like “16 in flight” doesn’t bound memory, and when memory runs out the engine preempts a request and later recomputes it, adding a second source of tail latency. Second, cost is partially predictable: output length can be estimated from the prompt, but with substantial error. Policies that depend on job size—like shortest-job-first or token budgets—inherit that error.

This paper focuses on the admission layer of an inference gateway: the component that decides which queued request gets dispatched next and whether any request may be dispatched at all. The central question is: when request cost is heavy-tailed and only partly predictable, which scheduling and admission policy best protects p99 latency?

The answer has a clean structure. Ordering by predicted size needs only coarse predictions—the tail is driven by separating short requests from very long ones, a distinction that survives large prediction error. Token budgets, by contrast, need accurate predictions, because every error becomes either idle budget or memory overflow.

Contributions.

- A queueing model of a gateway in front of continuous-batching engines: an $M/G/c$ queue with non-preemptive dispatch, truncated power-law service, and a per-backend KV budget (§3).
- Three policies and a hybrid, formally defined: FIFO with a count limit, SPJF with aging, and token-budget admission with a reserved short partition and age-updated charges based on the Pareto mean residual life (§4).
- Tail-scaling results and three design rules for prediction error: the misordering probability $\Phi\left(-\ lnr\ /\ (\sigma\sqrt{2})\right)$, a Poisson leakage limit for the protected partition, and a crossover σ^{*} for budgets (§4.4).
- A simulation study of five policies across load, predictor error, and tail index—including tuning-sensitivity ablations and an honest account of where the proposed hybrid does not win (§5–6).

Background and Related Work

Heavy-tailed queues

When service time S has a regularly varying tail with index α, the waiting time in an M/G/1 FIFO queue is regularly varying with index $\alpha - 1$—one order heavier than service itself ; for $1 < \alpha < 2$ its mean is infinite. Foss and Korshunov showed that in a c-server FIFO queue, the waiting-time tail is governed by how many servers large jobs must occupy before the remaining servers can no longer absorb the load. Harchol-Balter surveys the practical consequences for size-based scheduling: heavy tails concentrate load in the few largest jobs and favor policies that let small jobs bypass them.

Size-based scheduling with predictions

Shortest-job-first and its preemptive variant SRPT minimize mean response time when sizes are known exactly. Cobham derived the conditional waiting time under non-preemptive priorities—the result we use in §4.3. Mitzenmacher analyzed shortest-predicted-job-first in M/G/1 and bounded the cost of misprediction; Scully, Grosof, and Mitzenmacher extended this to uniform bounds under multiplicative estimate error. Those results concern mean response time on a single preemptible server; we need the p99 with multiple slots and no preemption.

LLM serving systems

Orca introduced iteration-level (continuous) batching. vLLM made KV memory paged, increasing the number of concurrent requests an engine can handle and making memory the binding resource. Sarathi-Serve chunks prefills to protect decode latency. DistServe disaggregates prefill and decode, and defines goodput as the fraction of requests meeting both a TTFT and a time-per-output-token (TPOT) target—the metric we adopt here. These systems schedule inside the engine; the gateway in front of them typically gets a fixed concurrency limit and nothing else.

Length prediction and admission control

Several systems predict output length to schedule LLM requests. Zheng et al. have the model estimate its own response length to batch similar-length requests together; S^3 predicts length to size KV allocations; Fu et al. learn a ranking rather than absolute lengths; FastServe sidesteps predictions entirely with a skip-join multi-level feedback queue that uses request age instead. Adaptive concurrency limits adjust the in-flight count based on latency gradients, but remain count-based. What's missing is an analysis that maps predictor error to p99 latency and to KV overflow at a non-preemptive gateway—that's the gap this paper addresses.

System Model

We consider one gateway dispatching requests to N_b inference backends. Each backend runs continuous batching, so within its concurrency limit each running request makes independent progress. Once dispatched, a request cannot be paused by the gateway; only the engine can preempt it, and only to reclaim memory. Table 1 summarizes notation.

Notation.

Symbol	Meaning
N_b	number of inference backends
P	prompt length (tokens)
L	output length (tokens)
$\hat{L}$	predicted output length
S	service time per request
a, b, d	fixed overhead, prefill cost/token, decode time/token
α	output-length tail index
x_m, M	minimum and maximum output length
K	KV-cache capacity per backend (tokens)
c	concurrency limit (slots per backend)
ρ	offered load
k^*	$\lceil c - \rho_{tot} \rceil$ spare slots
τ	short/long prediction threshold
β	fraction of budget reserved for long requests
B	token budget per backend
γ	aging coefficient
σ	log-normal predictor error scale
g	tokens generated so far (per request)
Λ	offered load of leaked long requests in short partition

Service.

A request with prompt P and output L occupies a slot for

$$S = a + bP + dL,$$

where a is fixed overhead, b is the prefill cost per prompt token, and d is the decode time per output token. Decode time grows with batch size on real hardware; the analysis treats d as fixed.

Heavy tail.

Output length follows a truncated power law,

$$Pr(L > x) = \left(\frac{x}{x_m}\right)^{-\alpha}, \quad x_m \le x \le M, \quad 1 < \alpha < 2.$$

The cap M keeps every moment finite, but when M is many times the median, the system behaves as heavy-tailed over the range that determines p99.

Memory.

A running request with g tokens generated holds $P + g$ KV tokens. If a backend's total exceeds K, the engine preempts its youngest request, which then waits and later pays a recompute of $b(P + g)$ before resuming. A count limit doesn't prevent this; a token budget can.

Metrics.

The policy-sensitive targets are p99 TTFT (reported separately for short [$L \le 256$] and long requests), p99 slowdown (end-to-end latency divided by the request's own service time), and *goodput*: the fraction of arrivals completed with $TTFT \le 2$ s and $TPOT \le 60$ ms. Rejected requests count against goodput, so no policy can improve its p99 by converting slow requests into errors. We also report p99 end-to-end latency, though for heavy-tailed outputs this mostly reflects long requests' own generation time—something no scheduler can remove.

Gateway model. Requests are ordered in the gateway queue and pass an admission test before dispatch; both highlighted stages are where the policies act. Under token-budget admission each backend's budget reserves a share $(1 - \beta)B$ *that predicted-long requests cannot hold. Requests still queued at their deadline are rejected.*

Policies and Analysis

Policies

Each policy is a pair: a queue order (who goes next) and an admission test (whether anyone may go now). Table 2 lists them.

Policies compared. All share the same deadline rejection.

Policy	Queue order	Admission test
FIFO-C	FIFO	count $\leq C$
SPJF	$\hat{S} - \gamma(t - t_{arr})$	count $\leq C$
TBA	FIFO	token budget B, partition β
Hybrid	$\hat{S} - \gamma(t - t_{arr})$	token budget B, partition β
LAS-charge	FIFO	age-only charges, no partition

SPJF with aging orders the queue by $\hat{S} - \gamma(t - t_{arr})$, where $\hat{S} = a + bP + d\hat{L}$ is the predicted cost. A positive γ bounds how long a long request can be passed over. Token-budget admission charges each in-flight request

$$c(g) = P + g + min\left(M - g,\ max\left(\hat{L}\, e^{\sigma z_q} - g,\ \frac{g}{\alpha - 1}\right)\right),$$

where z_q is the standard-normal q-quantile ($q = 0.9$). The second term inside the max is the Pareto mean residual life: under a power-law tail, a request that has already produced g tokens is expected to produce $g/(\alpha - 1)$ more, so charges grow for requests that outlive their prediction. A request is dispatched to the backend with the most free budget if its charge fits—and a predicted-long request ($\hat{L} > \tau$) only if the long share stays within βB. The queue is scanned first-fit, so a stuck long request doesn't block short ones behind it.

FIFO: spare slots set the tail

For one slot, the Pollaczek–Khinchine formula gives $E[W] = \lambda E[S^2] / (2(1 - \rho))$. With $1 < \alpha < 2$, the untruncated $E[S^2]$ is infinite, and with the cap it grows as $M^{2-\alpha}$—so raising `max_tokens` directly raises mean wait. For c slots, the result of gives

$$k^* = \lceil c - \rho_{tot} \rceil, \qquad Pr(W > x) \approx (x \cdot Pr(S > x))^{k^*} \sim x^{-k^*(\alpha-1)}.$$

Proposition 1 (Spare slots). *Under FIFO-C, the wait tail index is* $k^*(\alpha - 1)$*. Each whole slot of slack between load and concurrency adds* $\alpha - 1$ *to the tail index. A limit tuned to keep slots busy therefore removes slack one slot at a time—and each removal makes the tail a full step heavier.*

SPJF: ordering helps only with several slots

Non-preemptive priority on one slot gives Cobham's conditional wait for a job of size x :

$$E[W(x)] = \frac{\lambda E[S^2]/2}{(1-\rho_{<x})(1-\rho_{\leq x})}.$$

The numerator carries the same $E[S^2]$ blow-up as FIFO, so on one slot SPJF can't remove the heavy tail. With c slots, a short job jumps the queue and waits only for the first of c running jobs to finish.

Proposition 2 (Ordering gain, heuristic). *For predicted-short jobs,* $Pr(W > x)$ *decays roughly as* $x^{-c(\alpha-1)}$*, against* $x^{-k^*(\alpha-1)}$ *for FIFO-C. Since* $k^* \leq c$*, the gain is largest at high load, where* k^* *is small.*

TBA: isolation by a reserved partition

Proposition 3 (Isolation). *With exact predictions, predicted-short requests always have* $(1 - \beta)B$ *of budget that long requests cannot touch, so their wait matches that of a queue serving only short requests. Short service times are bounded by* τ*, making this wait light-tailed whenever short load fits the reserved budget. If charges never under-run true usage, the KV budget is never exceeded and the engine never preempts.*

Prediction error: three design rules

Model the predictor as $\widehat{lnL} = lnL + \sigma Z$ with $Z \sim N(0,1)$.

Rule 1 (Misordering).

Two requests with true length ratio $r > 1$ are swapped by SPJF with probability

$$Pr(misorder) = \Phi\left(\frac{-lnr}{\sigma\sqrt{2}}\right).$$

The p99 benefit of SPJF comes from keeping short requests ahead of tail requests, whose ratio to the median is large. At $r = 20$, a predictor with $\sigma = 1$ still orders 98.3% of such pairs correctly (Figure 2). SPJF should therefore be insensitive to σ up to about 1.5.

Probability that SPJF swaps two requests, from [eq:misorder]. Swaps between similar lengths are common at any realistic σ*, but swaps between a short request and a tail request (*$r \approx 20$*) stay rare up to* $\sigma = 1$*.*

Rule 2 (Leakage).

A long request of true length $k\tau$ is admitted to the short partition with probability $\Phi(-\ lnk\ /\ \sigma)$: 8.3% at $k\ =\ 4$ and $\sigma\ =\ 1$, but only 0.28% at $\sigma\ =\ 0.5$. The number of leaked requests in service is approximately Poisson with mean Λ, their offered load in slots. Reusing the k^* argument inside the partition, short requests are protected at the p99 if

$$Pr\left(Poisson(\Lambda)\ \geq k_s^*\right) < 0.01,$$

which allows Λ up to 0.010, 0.149, 0.436, and 0.823 for $k_s^*\ =\ 1, 2, 3$, and 4 spare short slots.

Rule 3 (Budget crossover).

Padding by $e^{\sigma z_q}$ keeps $Pr(L\ >\ charge)\ =\ 1\ -\ q$ but inflates the mean charge by

$$inflation(\sigma)\ =\ e^{\sigma z_q + \sigma^2/2}.$$

That's $1.42\ \times$ at $\sigma\ =\ 0.25$, $2.15\ \times$ at 0.5, $5.94\ \times$ at 1.0, and $21.1\ \times$ at 1.5. A KV-safe count limit must charge every request the full cap M, so static TBA only admits more work while $inflation(\sigma)\ <\ M/E[L]$. For the workload in §5, $M/E[L]\ =\ 38$ and the predicted crossover is $\sigma^*\ \approx\ 1.70$. Because operators set count limits well below the KV-safe value, the real crossover should be lower—§6.2 measures it.

Algorithm

The hybrid runs once per scheduling tick (Algorithm [alg:hybrid]). Charges are recomputed every tick from each request's current g, so the budget adjusts for requests that outlive their prediction.

Sort queue by $\hat{S}(req)\ -\ \gamma\ (now\ -\ req.arrival)$

Experimental Methodology

We built a time-stepped simulator in Python with 0.1 s resolution. It models two backends, each with an engine that runs up to 32 requests, holds a KV counter of $K\ =\ 6,000$ tokens, preempts its youngest request when the counter overflows, and admits waiting requests first-come first-served when memory frees up. Preempted requests keep their progress and pay a recompute of $b(P\ +\ g)$ on resumption. The gateway implements all five policies from Table 2.

Each configuration ran for 12,000 measured requests after 1,000 warm-up requests, with three random seeds; we report the mean and, where shown, the standard deviation across seeds. Table 3 lists parameters.

Simulation parameters.

Parameter	Value
Backends N_b	2
Engine concurrency	32 requests
KV capacity K	6,000 tokens
Tail index α (default)	1.5
Load ρ (default)	0.85
Short threshold τ	256 tokens
Goodput TTFT target	2 s
Goodput TPOT target	60 ms
Warm-up / measured requests	1,000 / 12,000
Random seeds	0, 1, 2
Aging coefficient γ	0.05
Budget fraction β (pilot)	0.3
Overcommit B/K (pilot)	1.5

Tuning the baselines fairly.

We swept the count limit over $\{6, 8, 12, 16, 24\}$ per backend for both FIFO-C and SPJF. Too low a limit starves the engines; too high a limit moves the queue into the engine's own first-come first-served wait list, where the gateway's ordering no longer applies. A limit of 12 was best or within 6% of the best p99 for FIFO-C at both $\rho = 0.7$ and 0.85, and for SPJF at 0.85; SPJF did 21% better with 8 at $\rho = 0.7$. We used 12 for both throughout. TBA's β and B/K were chosen from a 6,000-request pilot; §6.5 shows that a different setting does better, so the hybrid results are conservative.

Metrics.

P50 and p99 TTFT (all, short, and long requests), p99 slowdown, p99 end-to-end latency, goodput, rejection rate, and engine preemptions per 1,000 requests.

Results

Tail latency and goodput versus load

Figure 3 shows p99 TTFT for short requests and goodput as load increases. At $\rho = 0.5$, every policy except LAS-charge lands within about 1 s of the others: there's enough slack that ordering and admission barely matter, consistent with Proposition 1. From

$\rho = 0.7$ onward, the policies separate. The hybrid keeps short-request p99 TTFT below 4.3 s across the full range and is the only policy whose goodput stays above 95% at $\rho = 0.95$.

Effect of load (mean of three seeds; error bars ± 1 s.d.). Predicted-size policies use $\sigma = 0.5$. (a) p99 TTFT of short requests, log scale. (b) Goodput, counting rejections and late requests as failures.

Table 4 gives the full picture at $\rho = 0.85$. Against FIFO-C, the hybrid cuts short-request p99 TTFT $7.1\times$ (22.8 s to 3.2 s), overall p99 TTFT $3.4\times$, and p99 slowdown from 11.8 to 2.1. Engine preemptions drop from 2.79 to 0.05 per 1,000 requests—because the budget, not a count, controls admission. P99 end-to-end latency improves only 18%, as expected: it's dominated by the longest generations' own service time.

Results at $\rho = 0.85$, $\alpha = 1.5$ (predicted-size policies at $\sigma = 0.5$). TTFT in seconds; mean $\pm$ s.d. over three seeds.

Policy	p99 TTFT (short)	p99 TTFT (all)	p99 slowdown	Preemptions per 1k	Goodput (%)
FIFO-C	22.8	–	11.8	2.79	87.6
SPJF	9.5	–	4.9	2.61	92.5
TBA	7.1	–	3.7	0.12	91.8
Hybrid	3.2	–	2.1	0.05	96.7
LAS-charge	22.4	–	11.6	0.18	87.9

The components add up cleanly. SPJF alone gives a $2.4\times$ gain in short-request p99 through reordering; TBA alone gives $3.2\times$ through isolation and by eliminating preemptions; together they give $7.1\times$. Variance also shrinks: TBA and the hybrid have standard deviations a quarter of FIFO's or less, because the reserved partition stops a single burst of long requests from setting the p99 for everyone.

Robustness to prediction error

Figure 4 sweeps the predictor's log-error at $\rho = 0.85$. SPJF is essentially flat: its short-request p99 TTFT stays between 9.3 s and 12.8 s, and goodput between 92.5% and 92.9%, from $\sigma = 0$ to 1.5—confirming Rule 1. Budget-based policies are not flat. The hybrid holds 3.1–3.6 s up to $\sigma = 0.5$, then rises to 10.6 s at $\sigma = 1$ (no better than SPJF alone), and reaches 20.4 s at $\sigma = 1.5$. TBA alone falls below FIFO-C's goodput between $\sigma = 0.5$ and 1.0 (79.3% at $\sigma = 1$ against FIFO-C's 87.6%), and at $\sigma = 1.5$ it rejects 9.7% of arrivals.

Effect of predictor error at $\rho = 0.85$. The dash-dot line is FIFO-C, which uses no predictions. SPJF is insensitive to σ; both budget-based policies degrade beyond $\sigma = 0.5$ as padding inflates every charge.

The measured crossover falls well below Rule 3's $\sigma^{*} \approx 1.70$. The rule compares budgets with a KV-safe count limit that charges every request the full cap; the tuned limit of 12 is far from KV-safe and accepts a few preemptions per 1,000 requests in exchange for much more concurrency. Rule 3 is therefore an upper bound—the practical threshold for this workload is $\sigma \approx 0.5$–0.75. A production operator should measure their predictor's σ on held-out traffic before deciding whether to deploy budgets.

Effect of tail index

Table 5 varies the tail index at $\rho = 0.85$. The heavier the tail, the more the policies matter. At $\alpha = 1.9$, FIFO-C already meets the p99 target and the hybrid gains 1.5 s. At $\alpha = 1.2$, FIFO-C's goodput collapses to 62.7% with 17.8 preemptions per 1,000 requests; SPJF recovers only to 77.8% because it does nothing about memory; and the hybrid holds 93.0%.

Effect of tail index α at $\rho = 0.85$, $\sigma = 0.5$. P99 TTFT of short requests (seconds), goodput in parentheses.

Policy	$\alpha = 1.2$	$\alpha = 1.5$	$\alpha = 1.9$
FIFO-C	– (62.7%)	22.8 (87.6%)	– (–)
SPJF	– (77.8%)	9.5 (92.5%)	– (–)
Hybrid	– (93.0%)	3.2 (96.7%)	– (–)

Where the hybrid does not win

Four results cut against the proposal. First, long requests pay: at $\rho = 0.85$ their p99 TTFT is 24.5 s under the hybrid versus 20.1 s under FIFO-C (22% higher), and at $\alpha = 1.9$ it's 14.6 s versus 3.4 s. Aging with $\gamma = 0.05$ bounds but doesn't eliminate this. Second, SPJF's gain is smaller than we hypothesized: $2.4\times$ rather than the $5\times$ we expected, because with a count limit of 12 the engine's own FIFO wait list absorbs part of the reordering benefit. Third, the prediction-free ablation fails: LAS-charge, which uses only request age, is indistinguishable from FIFO-C (22.4 s versus 22.8 s). Without a predictor there's no partition and no ordering, and age-based charges alone only cap memory. Fourth, the hybrid stops paying for itself past $\sigma \approx 1$: at $\sigma = 1.5$ its goodput (82.3%) falls below both SPJF (92.8%) and FIFO-C (87.6%).

Ablation: budget settings and the starvation guard

Table 6 varies the long share β and the overcommit B/K at $\rho = 0.85$ and $\sigma = 0.5$. A strict budget ($B = K$)—which never lets charges exceed physical memory—is the worst setting across every β: the mean-residual charge over-reserves for requests that will finish soon, and goodput falls to 58–93% across TBA and the hybrid. Overcommitting to 1.5–$2\times K$ recovers concurrency at the cost of a few preemptions. The best hybrid setting found, $\beta = 0.5$ and $B = 2K$, gives 2.2 s and 97.7% goodput—better than the numbers in §6.

Sensitivity of the hybrid to β and B/K at $\rho = 0.85$, $\sigma = 0.5$. P99 TTFT of short requests (seconds), goodput in parentheses.

B/K	$\beta = 0.2$	$\beta = 0.3$	$\beta = 0.5$	$\beta = 0.7$
1.0	–	–	–	–
1.5	–	3.2 (96.7%)	–	–
2.0	–	–	2.2 (97.7%)	–

A starvation guard that stops scanning the queue once the oldest request has waited 20 s is harmless for the hybrid (3.3 s versus 3.2 s) but destructive for TBA alone (28.1 s versus 7.2 s, goodput 74.0% versus 91.8%). Under FIFO order the guard turns one stuck long request into head-of-line blocking for everyone; under SPJF order the stuck request rarely sits at the head. Any starvation guard should be paired with size-based ordering—or replaced by aging.

Discussion

What to deploy.

The results separate cleanly by what the operator knows. With no length predictor, the best available improvement is tuning the count limit: in our sweep, moving from 6 to 12 per backend cut p99 TTFT by 1.6–$2.8\times$. With a coarse predictor (σ around 1 or worse), SPJF with aging is the safe choice: it gains $2.4\times$ and is insensitive to error. With a good predictor ($\sigma \leq 0.5$), the hybrid adds a further $3\times$ and eliminates nearly all engine preemptions. Measuring σ on held-out traffic is therefore the first step—and it's cheap.

Validation in a real gateway.

These results come from a simulator that assumes Poisson arrivals, a fixed decode rate per slot, and an idealized engine. The next step is to implement the policies behind the admission interface of the Bulwark gateway (Project 2), replay production-shaped traces (the Azure LLM inference trace and BurstGPT) open-loop, and run them against a backend emulator calibrated to `llama.cpp`—with one real-backend run per policy to validate the emulator. The simulator's predictions are the hypotheses those runs will test: a $7\times$ short-request p99 gain at $\sigma = 0.5$, insensitivity of SPJF to σ, and a budget crossover near $\sigma = 0.5$–0.75.

Link to error budgets.

Admission converts latency into availability: every request rejected to protect the p99 is a failed request against an availability SLO. Under a 99.5% target the error budget is 0.5% of requests, so a policy that rejects 0.2% during peaks spends 40% of the monthly budget on admission alone. The companion paper on burn-rate alerting governs how much of that is affordable. A natural extension is to make the deadline threshold budget-aware—rejecting less when the availability burn rate is already high.

Threats to validity.

The engine's own scheduler can undo gateway ordering, which is why the count limit matters; real engines also use chunked prefill and prefix caching, which change b per request. The log-normal error model is symmetric and light-tailed, while real predictors are biased and occasionally wildly wrong—replaying a real predictor's empirical errors would stress-test Rules 1–3 more rigorously. Finally, three seeds of 12,000 requests give roughly 120 samples above each p99: enough to rank the policies, but not enough to resolve differences smaller than about 20%.

Conclusion and Future Work

Heavy-tailed request costs make an inference gateway's admission policy a first-order factor in tail latency. Queueing theory explains why: FIFO's tail is set by how many slots long jobs can block, so spare slots and protected capacity matter more than raw throughput. Prediction error splits the policies cleanly in two. Ordering by predicted size only needs to distinguish short from very long requests—a distinction that survives large errors and stays robust. Token budgets turn every error into wasted or overflowing memory and stop paying off once the log-error passes about 0.5–1.

In simulation, combining the two cuts short-request p99 TTFT $7.1\times$ and raises goodput from 87.6% to 96.7% at 85% load—at the cost of 22% higher p99 TTFT for long requests.

Future work is to validate these results in the Bulwark gateway under replayed traces, to measure a real length predictor's error distribution, to make the budget adaptive so B/K tracks observed preemptions, and to couple rejection behavior to the availability error budget.

Reproducibility

The simulator (`sim.py`), experiment drivers (`grid.py`, `abl.py`, `tune2.py`), and figure script (`figs.py`) accompany this paper. A 12,000-request run takes 1–3 s on one core; the main grid of 222 runs completes in under two minutes on two cores, and the ablation in about one minute. Seeds 0–2 were used throughout. The count-limit sweep and the budget pilot used 6,000 requests and two seeds.

A. G. Pakes. On the tails of waiting-time distributions. *Journal of Applied Probability*, 12(3):555–564, 1975.

S. Foss and D. Korshunov. Heavy tails in multi-server queue. *Queueing Systems*, 52(1):31–48, 2006.

S. Foss and D. Korshunov. On large delays in multi-server queues with heavy tails. *Mathematics of Operations Research*, 37(2):201–218, 2012.

M. Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

A. Cobham. Priority assignment in waiting line problems. *Journal of the Operations Research Society of America*, 2(1):70–76, 1954.

M. Mitzenmacher. Scheduling with predictions and the price of misprediction. In *Proc. ITCS*, 2020.

Z. Scully, I. Grosof, and M. Mitzenmacher. Uniform bounds for scheduling with job size estimates. In *Proc. ITCS*, 2022.

G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *Proc. USENIX OSDI*, 2022.

W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proc. ACM SOSP*, 2023.

A. Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *Proc. USENIX OSDI*, 2024.

Y. Zhong et al. : Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proc. USENIX OSDI*, 2024.

Z. Zheng et al. Response length perception and sequence scheduling: An LLM-empowered LLM inference pipeline. In *Proc. NeurIPS*, 2023.

Y. Jin et al. : Increasing GPU utilization during generative inference for higher throughput. In *Proc. NeurIPS*, 2023.

Y. Fu et al. Efficient LLM scheduling by learning to rank. In *Proc. NeurIPS*, 2024.

B. Wu et al. Fast distributed inference serving for large language models. arXiv:2305.05920, 2023.

P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *Proc. ISCA*, 2024.

Y. Wang et al. : A real-world workload dataset to optimize LLM serving systems. arXiv:2401.17644, 2024.

Netflix. concurrency-limits: Java library that implements and integrates concepts from TCP congestion control to auto-detect concurrency limits. GitHub repository, 2020.

S. Babu. Detection time versus false alerts in multi-window burn-rate alerting. Companion paper, ArdentMind, 2026.